

Building a Top-5 Tournament AI

Citadel Terminal Algorithmic Competition. I beat over 1,000 applicants and finished in the top five.

Top 5
out of 1,000+

Built a fully search-driven agent that competes against teams of engineers from world-class firms. No heuristics. Every move on every turn is selected by simulating thousands of futures and picking the one with the highest expected outcome.

What This Took

Citadel Terminal is a real-time strategy game where two agents face off on a 28 by 28 grid. Each turn an agent has fifteen seconds to decide how to spend two limited resources across thousands of possible actions: build structures, upgrade them or send mobile attackers. The official Java engine simulates the match. The surviving agent wins. Algorithms compete on a live ELO ladder against teams from the strongest firms in the industry.

Most submissions are hand-tuned heuristics. I went the other way and designed an agent where every decision is the output of a search loop. On every turn the agent generates roughly 2,500 candidate plans, samples plausible opponent responses from a model trained on 427 ranked replays, rolls each candidate forward through a fast simulator and commits to the plan with the highest expected utility. No part of this was scripted. The agent rediscovers the right opening, mid-game and end-game strategy from first principles every match.

The Headline Engineering Win

Search-driven play only works if the simulator is fast enough to roll out thousands of futures inside a fifteen-second budget. The official Java engine could not be embedded. The reference Python simulator ran at 336 games per second. I needed a hundred times that.

I built one from scratch. **sim_rs** is a complete reimplement of the Citadel action phase in Rust, exposed to the Python search loop as a compiled extension. It runs on any platform with no subprocess overhead. Reaching parity required decompiling the engine bytecode to recover the exact frame ordering and targeting priority. The result is below.

14.3K	Games per second on a single core. About 43 times faster than the reference Python sim.
75K	Games per second across 10 threads. About 225 times faster.
13,319 / 13,319	Fuzz tests where the Rust output is byte-identical to the engine. Zero divergence.
42 / 42	Rust unit tests passing.

Reading the Opponent in Real Time

An agent that ignores who it is playing leaves wins on the table. Mine watches the opponent every turn and adapts:

- **Opponent classifier.** Each turn the agent looks at the opponent's spawn pattern over a rolling ten-turn window. It computes spawn-tile concentration and unit-type diversity to decide whether the opponent is drilling one tile every turn (a vulnerability my defense can exploit) or playing a balanced game (where the wrong specialization would lose). The result gates the rest of the adaptive system so a wrong read never causes a regression.
- **Funnel detector.** It tracks the tiles where the opponent has been breaching me. When a single tile receives the majority of recent breaches the agent injects defense templates clustered around that exact tile into the search candidate pool. The search then picks among them on the merits.

Closing the Loop on Every Loss

The thing I am most proud of is the diagnosis pipeline. Every time my agent lost a ranked match I pulled the full replay from the competition's authenticated REST API through a custom browser-driven tool, walked the thousands of frames turn by turn and produced a structured account of the loss. I could see exactly which turn the agent fell behind, which plan it picked, what alternatives the search had considered and why it ranked them lower. I then replayed the same match against the new variant locally to prove the fix would have prevented the loss before shipping a single line of code.

This produced three successive variants. Each one is a documented response to a specific failure mode found in the previous one's losses.

<code>smart_oracle_vd</code>	First adaptive defense. Added the funnel detector and flank-corridor templates as a response to opponents who repeatedly attacked one side.
<code>smart_oracle_F</code>	Combined the funnel response with my strongest baseline. The first merge regressed against multi-archetype opponents. I diagnosed the cause from the replays and added a classifier as a safety gate so specialized defense only fires when the opponent is provably vulnerable to it. No regression on prior wins.
<code>smart_oracle_F2</code>	Replay analysis of four live losses revealed the funnel detector was firing far too late. It only triggered after the agent had already lost twenty-five health points. I lowered the trigger threshold, taught the detector to recognize a new attack vector it had been blind to and added defense templates clustered around the exact tiles where the losses happened.

The Habits That Made the Result Possible

- **Every change was justified by data.** I never shipped an idea I could not point to a replay or a benchmark to defend.
- **Backward-compatible by design.** Every new behavior was gated on a verifiable runtime signal. When the gate is closed the agent is byte-identical to its predecessor. The risk of any single change was bounded.
- **Local benchmark battery as a regression suite.** Every variant ran against a fixed opponent set on both spawn sides before shipping. A variant only shipped if it kept every prior win.
- **Honest failure analysis.** When my first merge regressed I did not paper over it. I documented it openly, traced the root cause through the replays and shipped the actual fix in the next variant.

Outcomes

- **Top five out of more than 1,000 applicants** on the official live ladder.
- **Twelve wins, zero losses on the local benchmark** covering my own earlier variants, the strongest published competitor finalists from Lostkids, Midwest and Georgia Tech and decoded ranked-match attackers.
- **Roughly 50,000 lines of code** across the Rust simulator, the Python search, the opponent-modeling pipeline, the replay analysis tooling and the benchmark harnesses. End-to-end ownership of architecture, implementation, validation and live-ladder iteration.
- Every claim above is verifiable. The code, performance benchmarks, parity fuzz reports and per-loss replay analyses are all preserved.